

CS554 Final Project Report

Manh Tien Nguyen

May 20, 2026

1 Introduction

This project investigates the NVIDIA PTX memory model by implementing classical weak-memory litmus tests as inline PTX inside CUDA kernels. We study four standard patterns:

- **Store Buffering (SB):** tests store-load reordering
- **Message Passing (MP):** tests store-store / load-load ordering
- **Load Buffering (LB):** tests causality cycles (load-store reordering)
- **Independent Reads of Independent Writes (IRIW):** tests multi-copy atomicity

For each test we compare multiple semantic variants: **WEAK** (plain loads/stores), **RELAXED** at CTA/GPU/SYS scope, **RELEASE/ACQUIRE**, and **FENCE SC**. Each variant is run $5 \times 1\,000\,000$ iterations (5 M total observations per test) on a **Tesla T4 (sm_75, Turing, CUDA 12.8)**.

Goals:

1. Observe which weak behaviors actually occur on real hardware.
2. Compare results to the PTX ISA specification and the formal model of Lustig et al. [1].
3. Identify and fix methodology bugs that produce spurious results.
4. Introduce multi-run statistics and full outcome histograms for publication-quality evidence.

2 Implementation

The code is located at <https://github.com/manhntm3/CS554FinalProject>.

I implemented four canonical litmus test patterns as three separate CUDA kernels. To isolate hardware behavior and avoid compiler-induced instruction reordering, all memory operations were executed using **volatile inline PTX assembly**. The implementation includes options for inter-block and intra-block configuration, as Nvidia forces memory instruction based on scope (.gpu and .sys).

2.1 Array-Strided Memory Layout

A critical design choice is the **array-strided layout**: each of the N iterations accesses a unique pair of memory locations $x[i]$ and $y[i]$, pre-initialized to 0 by the host. This eliminates cross-iteration L1 cache pollution, which caused the false weak behaviors in the first implementation (see Section 3).

2.2 PTX Primitives

2.3 SASS Verification

Compiling with `-Xptxas -O0` prevents PTXAS from reordering instructions [2]. The key PTX-to-SASS mappings on sm_75 are:

All inline PTX instructions survive the compilation pipeline verbatim and no merging or re-ordering was observed.

WEAK	RELAXED GPU
<pre>// P0: Store X, Load Y st.global.u32 [x], 1; r0 = ld.global.u32 [y];</pre>	<pre>// P1: Store Y, Load X st.global.relaxed.gpu.u32 [x], 1; r0 = ld.global.relaxed.gpu.u32 [y];</pre>

Figure 1: Store Buffering (SB) Litmus Test PTX: Weak vs Relaxed GPU-scope

RELEASE/ACQUIRE GPU	FENCE SC GPU
<pre>// P0 (Producer) st.global.relaxed.gpu.u32 [data], 1; st.global.release.gpu.u32 [flag], 1; // P1 (Consumer) r_flag = ld.global.acquire.gpu.u32 [flag]; r_data = ld.global.relaxed.gpu.u32 [data];</pre>	<pre>// P0 (Producer) st.global.relaxed.gpu.u32 [data], 1; fence.sc.gpu; st.global.relaxed.gpu.u32 [flag], 1; // P1 (Consumer) r_flag = ld.global.relaxed.gpu.u32 [flag]; fence.sc.gpu; r_data = ld.global.relaxed.gpu.u32 [data];</pre>

Figure 2: Message Passing (MP) Litmus Test: Release/Acquire vs Fence SC

3 Methodology: Bugs Found and Fixed

3.1 First Try: Scalar Reuse (Flawed)

The initial MP implementation reused a single pair of scalars (`data`, `flag`) across iterations with a plain-C reset and a software spin-barrier:

```
// P0 reset (WRONG):
*data = 0; *flag = 0;
__threadfence(); // flushes P0's L1, but does NOT force P1 to invalidate its L1
global_spin_barrier(sync_barrier, threshold);
```

Root cause: `__threadfence()` flushes P0's SM to L2, but the consumer (P1, on a different SM) is not required to invalidate its own L1 cache for `data` and `flag`. The software spin barrier does not establish a *happens-before* edge between P0's scoped stores and P1's subsequent scoped loads under the PTX memory model. As a result, P1 could exit the barrier still holding a stale `flag=1` from iteration $i-1$, then read `data=0` from P0's reset, which is a false MP weak outcome.

The effect was severe: GPU-scope FENCE SC showed **24.57%** weak behavior (worse than RELAXED), which is physically impossible under a correct test.

The SB test, which already used an array-strided layout, did not exhibit this problem.

3.2 Second Try: Fixes Applied

All experiments in Section 4 use:

1. **Array-strided layout for all tests:** each iteration uses a fresh, pre-zeroed memory slot.
2. **Multi-run statistics:** 5 runs $\times$ 1M iterations; mean and standard deviation reported.
3. **Full outcome histograms:** all (r_0, r_1) pairs counted, not just the weak outcome.

4 Experimental Results

Hardware: Tesla T4 (sm_75, Turing) | CUDA 12.8 | 5 runs $\times$ 1 000 000 iter = 5 000 000 total obs per test.

Table 1: PTX to SASS mapping (sm_75 / Tesla T4)

PTX Instruction	SASS Instruction
st.global.u32 (weak)	STG.E
st.global.relaxed.gpu.u32	STG.E.STRONG.GPU
st.global.release.gpu.u32	STG.E.STRONG.GPU
ld.global.acquire.gpu.u32	LDG.E.STRONG.GPU
st.global.relaxed.sys.u32	STG.E.STRONG.SYS
fence.sc.cta	MEMBAR.ALL.CTA
fence.sc.gpu	MEMBAR.SC.GPU
fence.sc.sys	MEMBAR.SC.SYS
atom.global.relaxed.gpu.exch	ATOMG.E.EXCH.STRONG.GPU

Table 2: MP scalar implementation (first try, 5M iterations)

Variant	Weak Count	Weak %
Inter-Block WEAK	0	0.00%
Inter-Block RELAXED (GPU)	14,669	0.29%
Inter-Block ACQ/REL (GPU)	726,303	14.53%
Inter-Block FENCE SC (GPU)	973,503	19.47% ← impossible
Inter-Block FENCE SC (SYS)	0	0.00%

4.1 Store Buffering (SB)

Table 3: SB results: P0: st(x,1); r0=ld(y) P1: st(y,1); r1=ld(x) Weak: r0=0, r1=0

Variant	Weak	%	Std%	Outcome distribution (%)			
				(0,0)*	(0,1)	(1,0)	(1,1)
Inter-Block WEAK	826	0.0165	0.0024	0.016	99.983	0.000	0.000
Inter-Block RELAXED (GPU)	0	0.0000	0.0000	0.000	87.408	0.227	12.365
Inter-Block RELAXED (SYS)	0	0.0000	0.0000	0.000	85.902	0.255	13.843
Inter-Block RELAXED (CTA)	813	0.0163	0.0005	0.016	99.984	0.000	0.000
Intra-Block RELAXED (CTA)	0	0.0000	0.0000	0.000	24.131	18.429	57.441
Inter-Block ACQ/REL (GPU)	0	0.0000	0.0000	0.000	24.381	43.980	31.639
Inter-Block FENCE SC (GPU)	0	0.0000	0.0000	0.000	0.712	0.000	99.288
Inter-Block FENCE SC (SYS)	0	0.0000	0.0000	0.000	9.808	0.000	90.191

Key findings:

- **WEAK and RELAXED (CTA)** show identical weak rates ($\approx 0.016\%$), confirming that plain stores and CTA-scoped atomics provide no cross-SM ordering guarantee.
- **RELAXED (GPU/SYS)** show 0% weak, even though the PTX spec allows weak SB for relaxed operations. The T4’s L2 cache acts as a unified coherence point, enforcing GPU-wide write ordering even for relaxed atomics. This is *hardware stronger than spec*.
- **ACQ/REL (GPU)** shows 0% weak. Release/acquire prevents store-store and load-load reordering; the spec *does* allow SB here (store-load reordering), but the hardware is again conservative.
- **FENCE SC (GPU/SYS)** correctly eliminate all weak behavior (0%).
- The outcome distribution reveals strong scheduling bias: **WEAK** produces almost exclusively (0,1) (99.98%), meaning P0 consistently runs ahead of P1. The rare (0,0) outcomes are genuine reorderings.

4.2 Message Passing (MP)

Table 4: MP results (array-strided): P0: data=1; flag=1 P1: r_f=ld(flag); r_d=ld(data)
Weak: r_f=1, r_d=0

Variant	Weak	%	Std%	Outcome distribution (%)			
				(0,0)	(0,1)	(1,0)*	(1,1)
Inter-Block WEAK	0	0.0000	0.0000	0.001	0.000	0.000	99.999
Intra-Block RELAXED (CTA)	0	0.0000	0.0000	0.000	0.000	0.000	100.000
Inter-Block RELAXED (GPU)	0	0.0000	0.0000	0.000	0.000	0.000	99.999
Inter-Block RELAXED (SYS)	0	0.0000	0.0000	0.000	0.000	0.000	100.000
Intra-Block ACQ/REL (CTA)	0	0.0000	0.0000	0.000	0.000	0.000	100.000
Inter-Block ACQ/REL (GPU)	0	0.0000	0.0000	0.000	0.000	0.000	99.999
Inter-Block ACQ/REL (SYS)	0	0.0000	0.0000	99.9999	0.000	0.000	0.000
Intra-Block FENCE SC (CTA)	0	0.0000	0.0000	0.000	100.000	0.000	0.000
Inter-Block FENCE SC (GPU)	0	0.0000	0.0000	0.000	0.000	0.000	99.999
Inter-Block FENCE SC (SYS)	0	0.0000	0.0000	0.000	0.001	0.000	99.999

Key findings:

- **Zero MP weakness across all variants** after the array-layout fix. Even plain **WEAK** stores show 0% weak: the T4 enforces in-program-order store visibility from a single thread (a property at least as strong as TSO store-order), so the flag write never becomes visible before the data write.
- **ACQ/REL (SYS) shows 99.999% (0,0) outcomes**: the consumer almost always reads both values as 0. This is a timing effect and SYS-scope operations route through the host memory path and are significantly slower, so the consumer consistently reads both locations before the producer completes.
- **FENCE SC (CTA) intra-block** shows 100% (0,1): the consumer always sees **flag=0**, **data=1**. Within a single block, intra-warp execution order means the consumer runs *before* the producer writes the flag (data store already visible via L1 sharing, but flag store hasn't executed yet).

4.3 Load Buffering (LB)

Table 5: LB results (corrected): P0: r0=ld(y); st(x,1) P1: r1=ld(x); st(y,1) Weak: r0=1, r1=1

Variant	Weak	%	Std%	Outcome distribution (%)			
				(0,0)	(0,1)	(1,0)	(1,1)*
Inter-Block WEAK	0	0.0000	0.0000	89.104	2.797	8.099	0.000
Inter-Block RELAXED (GPU)	0	0.0000	0.0000	1.798	17.141	81.061	0.000
Inter-Block RELAXED (SYS)	0	0.0000	0.0000	1.707	16.867	81.427	0.000
Inter-Block ACQ/REL (GPU)	0	0.0000	0.0000	0.008	0.000	99.992	0.000
Inter-Block FENCE SC (GPU)	0	0.0000	0.0000	0.009	0.000	99.992	0.000

Key findings:

- **Zero LB weakness (r0=1, r1=1) across all 25M observations**. A causality cycle requires both threads to see each other's write simultaneously, which would violate the acyclicity of the happens-before relation. The PTX formal model [1] explicitly forbids out-of-thin-air (OOTA) values; these results confirm the T4 respects this axiom.
- **The distribution is now non-trivial and informative**, unlike the previous implementation: **RELAXED (GPU)** shows 81% (1,0) . P1 (block 1) tends to run slightly ahead, so P0 reads P1's write of $y = 1$ ($r_0 = 1$) while P1 has already read $x = 0$ ($r_1 = 0$) before P0 wrote it.

- **ACQ/REL and FENCE SC** show 99.99% (1, 0): the acquire/fence semantics cause the operations to be well-separated in time, strongly biasing P0 to see P1’s write but not vice versa.

4.4 Independent Reads of Independent Writes (IRIW)

Table 6: IRIW results: P0:x=1; P1:y=1; P2:r0=ld(x),r1=ld(y); P3:r2=ld(y),r3=ld(x). Weak: r0=1,r1=0,r2=1,r3=0

Variant	Weak	%	Std%	P2 (ld x→y) %				P3 (ld y→x) %			
				(0,0)	(1,0)*	(0,1)	(1,1)	(0,0)	(1,0)*	(0,1)	(1,1)
WEAK	0	0.0000	0.0000	0.0	0.0	0.0	100.0	0.0	0.0	0.0	100.0
RELAXED (GPU)	0	0.0000	0.0000	0.0	0.0	0.0	100.0	0.0	0.0	0.0	100.0
ACQ/REL (GPU)	0	0.0000	0.0000	0.0	0.0	0.0	100.0	0.0	0.0	0.0	100.0
FENCE SC(GPU)	0	0.0000	0.0000	0.0	0.0	0.0	100.0	0.0	0.0	0.0	100.0
FENCE SC(SYS)	0	0.0000	0.0000	0.0	0.0	0.0	100.0	0.0	0.0	0.0	100.0

Key findings:

- **Zero IRIW weakness across all variants.** Multi-copy atomicity is empirically confirmed on the Tesla T4 for all tested scope levels, including plain **WEAK** stores.
- **Both readers see (1,1) $\approx$ 100% of the time:** by the time the reader blocks reach iteration i , both writer blocks have typically already written $x[i] = 1$ and $y[i] = 1$. This is a timing effect of the array-strided layout: readers tend to lag slightly behind writers at this iteration count.
- These results confirm that **the T4’s L2 cache provides unified coherence**, ensuring writes propagate atomically to all SMs simultaneously. This is consistent with the PTX guarantee that all GPU-scope non-weak operations are multi-copy atomic, and agrees with prior empirical findings by Alglave et al. [3].

5 Challenges and Lessons Learned

1. **Hardware is stronger than the specification.** NVIDIA hardware enforces implicit ordering beyond what PTX specifies, particularly for GPU-scope relaxed atomics. This provides safety margins for programmers but also means that code that “works” on one GPU may fail on future hardware that implements the spec more faithfully.
2. **Fences are hardware intrinsics; barriers are software constructs.** Only hardware fences (`fence.sc.gpu`, `MEMBAR.SC.GPU`) establish the ordering semantics required to prevent weak behaviors at GPU scope. Software spin-barriers, even with `__threadfence()`, are insufficient because they don’t establish the right happens-before edges across SM boundaries.
3. **The array-strided layout is essential for correctness.** Scalar reuse with per-iteration resets interacts poorly with the multi-level GPU cache hierarchy and produces artifacts that look like weak memory behavior but are actually cache-coherence bugs in the test harness itself.
4. **Full outcome histograms are mandatory for meaningful results.** Reporting only the weak count obscures timing biases, reveals nothing about non-weak outcomes, and cannot detect a trivially broken test (as was the case with the original LB kernel).
5. **Multi-run statistics confirm reproducibility.** With stddev $\approx$ 0.002% for SB WEAK, the result is clearly reproducible and not noise. All zero-weak results show stddev = 0.0000% across all 5 runs.

References

- [1] D. Lustig, S. Sahasrabudhe, and O. Giroux, “A formal analysis of the nvidia ptx memory consistency model,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 257–270. [Online]. Available: <https://doi.org/10.1145/3297858.3304043>
- [2] NVIDIA Corporation, “PTX ISA reference: CUDA toolkit documentation,” 2024, version 8.5, accessed May 2025. [Online]. Available: <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html>
- [3] J. Alglave, M. Batty, A. F. Donaldson, G. Gopalakrishnan, J. Ketema, D. Poetzl, T. Sorensen, and J. Wickerson, “Gpu concurrency: Weak behaviours and programming assumptions,” in *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’15. New York, NY, USA: Association for Computing Machinery, 2015, pp. 577–591. [Online]. Available: <https://doi.org/10.1145/2694344.2694391>